

VendorBridge MySQL Schema

The following MySQL DDL recreates the PostgreSQL schema for the VendorBridge ERP, preserving all primary keys, foreign keys, and constraints. We replace PostgreSQL's `SERIAL` with `INT AUTO_INCREMENT` (equivalent behavior [14†L134-L142]) and use MySQL syntax for defaults and triggers. MySQL 8+ is assumed so we can use `DEFAULT (CURRENT_DATE)` on `DATE` columns [11†L599-L607]. Triggers are defined with the `CREATE TRIGGER ... BEFORE INSERT` syntax [12†L120-L127] to generate the `po_number` and `invoice_number` fields.

Roles Table

```
CREATE TABLE roles (  
  role_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL UNIQUE  
);
```

- `role_id` is the primary key (auto-incremented integer).
- `name` is a unique role name (e.g. "Procurement Officer", "Vendor", etc.).

Vendor Categories Table

```
CREATE TABLE vendor_categories (  
  category_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE  
);
```

- `category_id` is the primary key.
- `name` is a unique category name (e.g. "Electronics", "Office Supplies").

Vendors Table

```
CREATE TABLE vendors (  
  vendor_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(200) NOT NULL UNIQUE,  
  category_id INT,  
  gst_number VARCHAR(50) UNIQUE,  
  contact_name VARCHAR(100),  
  contact_email VARCHAR(255),  
  contact_phone VARCHAR(50),
```

```

address TEXT,
active BOOLEAN NOT NULL DEFAULT TRUE,
FOREIGN KEY (category_id) REFERENCES vendor_categories(category_id) ON DELETE
SET NULL
);

```

- `vendor_id` is the primary key.
- `category_id` references `vendor_categories(category_id)`. It is nullable so that `ON DELETE SET NULL` works if the category is deleted.
- Other fields store GST, contact info, etc.

Users Table

```

CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role_id INT NOT NULL,
  vendor_id INT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_login TIMESTAMP NULL,
  FOREIGN KEY (role_id) REFERENCES roles(role_id) ON DELETE RESTRICT,
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE SET NULL
);

```

- `user_id` is the primary key.
- `role_id` links to `roles(role_id)` (NOT NULL, RESTRICT delete).
- `vendor_id` links to `vendors(vendor_id)` (nullable, set to NULL if vendor is deleted).
- `password_hash` should store a secure hash (e.g. SHA2) of the password. In MySQL you can use functions like `SHA2()` or handle hashing in application code.

Sessions Table (Refresh Tokens)

```

CREATE TABLE sessions (
  session_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  refresh_token CHAR(36) NOT NULL UNIQUE,
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

```

- `refresh_token` is a UUID stored as a 36-character string (MySQL has no native UUID type).

- If a user is deleted, their sessions are automatically removed (`ON DELETE CASCADE`).

RFQs Table

```
CREATE TABLE rfqs (
  rfq_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  description TEXT,
  created_by INT NOT NULL,
  created_date DATE NOT NULL DEFAULT (CURRENT_DATE),
  deadline DATE NOT NULL,
  approved BOOLEAN NOT NULL DEFAULT FALSE,
  approved_by INT,
  FOREIGN KEY (created_by) REFERENCES users(user_id) ON DELETE RESTRICT,
  FOREIGN KEY (approved_by) REFERENCES users(user_id) ON DELETE SET NULL
);
```

- `rfq_id` is the primary key.
- `created_by` references `users(user_id)`.
- `created_date` defaults to the current date `【11†L599-L607】`.
- `approved_by` is nullable, referencing the user who approved the RFQ.

RFQ Lines Table

```
CREATE TABLE rfq_lines (
  rfq_line_id INT AUTO_INCREMENT PRIMARY KEY,
  rfq_id INT NOT NULL,
  line_no INT NOT NULL,
  item_description VARCHAR(500) NOT NULL,
  quantity INT NOT NULL CHECK (quantity > 0),
  unit VARCHAR(50),
  FOREIGN KEY (rfq_id) REFERENCES rfqs(rfq_id) ON DELETE CASCADE
);
```

- Each row represents one line item in an RFQ.
- `quantity` has a check constraint enforcing it to be positive (MySQL 8+ supports `CHECK`).

RFQ-Vendor Assignment Table

```
CREATE TABLE rfq_vendors (
  rfq_id INT NOT NULL,
  vendor_id INT NOT NULL,
  PRIMARY KEY (rfq_id, vendor_id),
  FOREIGN KEY (rfq_id) REFERENCES rfqs(rfq_id) ON DELETE CASCADE,
```

```
FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE CASCADE
);
```

- Junction table linking RFQs to assigned Vendors (many-to-many).
- Composite primary key `(rfq_id, vendor_id)` ensures each assignment is unique.

Quotations Table

```
CREATE TABLE quotations (
  quotation_id INT AUTO_INCREMENT PRIMARY KEY,
  rfq_id INT NOT NULL,
  vendor_id INT NOT NULL,
  quotation_date DATE NOT NULL DEFAULT (CURRENT_DATE),
  supplier_ref VARCHAR(100),
  notes TEXT,
  delivery_date DATE,
  selected BOOLEAN NOT NULL DEFAULT FALSE,
  FOREIGN KEY (rfq_id) REFERENCES rfqs(rfq_id) ON DELETE CASCADE,
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE CASCADE
);
```

- `quotation_date` defaults to the current date.
- `selected` marks whether this quotation was chosen.

Quotation Lines Table

```
CREATE TABLE quotation_lines (
  quotation_line_id INT AUTO_INCREMENT PRIMARY KEY,
  quotation_id INT NOT NULL,
  rfq_line_id INT,
  unit_price DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0),
  quantity INT NOT NULL CHECK (quantity > 0),
  promised_date DATE,
  FOREIGN KEY (quotation_id) REFERENCES quotations(quotation_id) ON DELETE CASCADE,
  FOREIGN KEY (rfq_line_id) REFERENCES rfq_lines(rfq_line_id) ON DELETE SET NULL
);
```

- Each row corresponds to a line item in a vendor's quotation.
- `rfq_line_id` is nullable in case the RFQ line is removed.
- Prices use `DECIMAL(12,2)` and have checks to ensure non-negative.

Purchase Orders Table

```
CREATE TABLE purchase_orders (  
  po_id INT AUTO_INCREMENT PRIMARY KEY,  
  quotation_id INT,  
  vendor_id INT NOT NULL,  
  created_by INT NOT NULL,  
  po_number VARCHAR(20) NOT NULL UNIQUE,  
  order_date DATE NOT NULL DEFAULT (CURRENT_DATE),  
  total_amount DECIMAL(14,2) NOT NULL DEFAULT 0,  
  approved BOOLEAN NOT NULL DEFAULT FALSE,  
  approved_by INT,  
  FOREIGN KEY (quotation_id) REFERENCES quotations(quotation_id) ON DELETE SET  
  NULL,  
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE RESTRICT,  
  FOREIGN KEY (created_by) REFERENCES users(user_id) ON DELETE RESTRICT,  
  FOREIGN KEY (approved_by) REFERENCES users(user_id) ON DELETE SET NULL  
);
```

- `po_number` is a generated field (format `P0000001` etc.).
- We will auto-generate `po_number` in a trigger (see below).
- Other fields reference the related quotation, vendor, and users.

Purchase Order Lines Table

```
CREATE TABLE purchase_order_lines (  
  po_line_id INT AUTO_INCREMENT PRIMARY KEY,  
  po_id INT NOT NULL,  
  quotation_line_id INT,  
  quantity INT NOT NULL CHECK (quantity > 0),  
  line_total DECIMAL(14,2) NOT NULL,  
  FOREIGN KEY (po_id) REFERENCES purchase_orders(po_id) ON DELETE CASCADE,  
  FOREIGN KEY (quotation_line_id) REFERENCES quotation_lines(quotation_line_id)  
  ON DELETE SET NULL  
);
```

- Each row is a line in a purchase order.
- `quantity` and `line_total` mirror the details from the quotation.

Invoices Table

```
CREATE TABLE invoices (  
  invoice_id INT AUTO_INCREMENT PRIMARY KEY,  
  po_id INT NOT NULL,
```

```

vendor_id INT NOT NULL,
created_by INT NOT NULL,
invoice_number VARCHAR(20) NOT NULL UNIQUE,
invoice_date DATE NOT NULL DEFAULT (CURRENT_DATE),
total_amount DECIMAL(14,2) NOT NULL DEFAULT 0,
status ENUM('Pending', 'Paid', 'Cancelled') NOT NULL DEFAULT 'Pending',
FOREIGN KEY (po_id) REFERENCES purchase_orders(po_id) ON DELETE CASCADE,
FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id) ON DELETE RESTRICT,
FOREIGN KEY (created_by) REFERENCES users(user_id) ON DELETE RESTRICT
);

```

- `invoice_number` is generated as `INV000001`, `INV000002`, etc. (trigger below).
- `status` is an `ENUM` for allowed values, with default `'Pending'`.
- Other columns reference the related PO, vendor, and user.

Invoice Lines Table

```

CREATE TABLE invoice_lines (
  invoice_line_id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT NOT NULL,
  po_line_id INT NOT NULL,
  quantity INT NOT NULL CHECK (quantity > 0),
  line_total DECIMAL(14,2) NOT NULL,
  FOREIGN KEY (invoice_id) REFERENCES invoices(invoice_id) ON DELETE CASCADE,
  FOREIGN KEY (po_line_id) REFERENCES purchase_order_lines(po_line_id) ON
DELETE RESTRICT
);

```

- Each row is one line on an invoice, linked to a purchase-order line.

Approvals Table

```

CREATE TABLE approvals (
  approval_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  rfq_id INT,
  po_id INT,
  approved BOOLEAN NOT NULL,
  remarks TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (rfq_id IS NOT NULL OR po_id IS NOT NULL),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE RESTRICT,
  FOREIGN KEY (rfq_id) REFERENCES rfqs(rfq_id) ON DELETE CASCADE,

```

```
FOREIGN KEY (po_id) REFERENCES purchase_orders(po_id) ON DELETE CASCADE
);
```

- An approval must be associated with *either* an RFQ or a PO (but not neither) as enforced by the check.
- `user_id` is the approver. Timestamps default to current time.

Activity Log Table

```
CREATE TABLE activity_log (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  entity VARCHAR(50) NOT NULL,
  entity_id INT NOT NULL,
  action TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE RESTRICT
);
```

- Immutable audit log of user actions (e.g. "User 5 created RFQ 12").
- `entity` might be "RFQ", "Quotation", etc., with the corresponding `entity_id`.

Triggers for Sequential PO and Invoice Numbers

MySQL triggers (using `CREATE TRIGGER ... BEFORE INSERT`) populate the `po_number` and `invoice_number` fields. The triggers below query the `information_schema.TABLES` to find the next `AUTO_INCREMENT` value. This is based on MySQL's trigger capabilities [\[12†L120-L127\]](#).

```
DELIMITER $$

CREATE TRIGGER trg_gen_po_number
BEFORE INSERT ON purchase_orders
FOR EACH ROW
BEGIN
  DECLARE next_id INT;
  SELECT AUTO_INCREMENT INTO next_id
  FROM information_schema.TABLES
  WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'purchase_orders';
  SET NEW.po_number = CONCAT('PO', LPAD(next_id, 6, '0'));
END$$

CREATE TRIGGER trg_gen_invoice_number
BEFORE INSERT ON invoices
FOR EACH ROW
BEGIN
```

```

DECLARE next_inv INT;
SELECT AUTO_INCREMENT INTO next_inv
  FROM information_schema.TABLES
  WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'invoices';
SET NEW.invoice_number = CONCAT('INV', LPAD(next_inv, 6, '0'));
END$$

DELIMITER ;

```

Each trigger runs *before* inserting a new row, calculates the next numeric ID, and prefixes it with “PO” or “INV” plus zero-padding to 6 digits. This mimics the PostgreSQL sequence-and-function approach in MySQL.

Notes:

- We assume MySQL 8.0+ for features like `DEFAULT (CURRENT_DATE)` 【11†L599-L607】 and native `CHECK` constraints.
- Password hashing is not built into SQL here; you can use MySQL’s `SHA2()` or manage hashing at the application level.
- All foreign keys and defaults match the intended workflow from the original PostgreSQL design, with appropriate `ON DELETE` actions to enforce referential integrity.

References: The table definitions use MySQL’s `AUTO_INCREMENT` syntax 【14†L134-L142】 and `CREATE TRIGGER` syntax 【12†L120-L127】 ; `DEFAULT (CURRENT_DATE)` on `DATE` columns is supported in MySQL 8.0+ 【11†L599-L607】 .